

Design and Evaluation of a WiFi-Based Intrusion Detection System for IoT Networks Using Suricata

Shravanthi Keshavamurthy - 319517
s.keshavamurthy@stud.fh-sm.de

Abstract

The increased number of IoT networks significantly augments the attack surface of current and future networks, and many of these networks, ranging from home networks and small-scale business environments to testing facilities, are being set-up without a monitoring security sentinel to protect them. The current intrusion detection systems are generally developed for enterprise networks and involve overhead infrastructure requirements beyond the needs of IoT environments. There is a pressing need for highly efficient intrusion detection systems to be developed to focus at the edge of the network. This paper describes the design and assessment of a WiFi-based intrusion detection system developed and deployed on a Raspberry Pi using the freely available Suricata intrusion detection engine. The new detection approach involves passively analyzing WiFi communications and using a set of defined signature-based detection rules that enable the detection of typical reconnaissance and scanning attacks such as ICMP sweeps, TCP SYN scans, and UDP scans. Triggered events are documented using the Suricata-formatted structured JSON logging functionality and analyzed by an alerting service written using Python that produces summarized email notifications to the network admin. The performance of this system was tested using attack scenarios developed with the tool Nmap. The experiment results confirm that the proposed system can detect network reconnaissance efficiently with low CPU usage for the Raspberry Pi system. This indicates that systems designed with open-source materials at the edge of a network can be a useful tool for improving network security because a typical network is unable to monitor network traffic using normal systems.

Keywords: Intrusion Detection System, IoT Security, WiFi Network Monitoring, Suricata, Raspberry Pi, Network Security, Edge Computing

1 Introduction

The Internet of Things (IoT) technologies have shown a tremendous pace of development and have gained widespread acceptance and usage. The IoT devices make it possible for the devices to be constantly controlled and automated. The IoT devices have played a pivotal role in forming the basis of smart living. Nonetheless, despite the tremendous growth of the Internet of Things devices, the growth has produced a substantial rise in the attack surfaces of the latest networks. The Internet of Things devices can be safely characterized as vulnerable devices.

One typical aspect of an IoT setup is the wireless connectivity, particularly in terms of Wi-Fi networks. Even though Wi-Fi networks enable flexibility, ease of setup, and mobility, IoT devices are vulnerable to passive observation and active probing by malicious parties within the Wi-Fi range. In home or small business networks, security is merely implemented on the router level. High-end security monitoring systems, for example, enterprise-class intrusion systems, are less commonly found in these networks due to issues related to expenses, complexity, and equipment needs. Therefore, very little is usually detected in terms of the initial stage actions by the attackers, which involve network observation or scanning.

Intrusion Detection Systems (IDS) are an essential component for detecting malicious activity based on network traffic and system patterns. Network intrusion detection systems are particularly well adapted for operating within the context of an Internet of Things network due to their ability to provide centralized monitoring without having to interface directly with the individual components on the network that are being monitored. The drawback to implementing an intrusion detection system is that there are usually expectations for high levels of processing that may not be present within an Internet of Things network.

Edge computing has recently been identified as a viable paradigm that helps in overcoming scalability, latency, and privacy issues in the context of IoT networks. Edge solutions help in overcoming the scalability, latency, and privacy issues in the context of IoT networks. In terms of security, using intrusion detection solutions in the context of the network edge helps in analyzing traffic patterns in close proximity to the IoT device, thus providing visibility while incurring low overhead in terms of deployment. Single-board computers, like Raspberry Pi, serve as a suitable solution for realizing the security solution using the edge paradigm because of their low power consumption, scalability, as well as adequate computation capabilities.

In this article, the design and analysis of a WiFi-based intrusion detection system for IoT networks will be discussed that employs the open-source Suricata intrusion detection system on the Raspberry Pi platform. The proposed system will work fully passively on the network as it will monitor the wireless traffic on the network and will use customized signature-based detection on the network for common reconnaissance maneuvers such as ICMP, TCP SYN, and UDP-based scanning. The alerts will be stored as structured JSON messages, which will then be processed for analysis of summarized alerts on an email service for the network administrator.

One major strength that emanates from this work is that it proves the capability to effectively secure network activities on WiFi IoT networks using cheap hardware and open-source solutions. Through rigorous scanning experiments, the proposed system is capable of detecting network reconnaissance with high accuracy and low complexity. This work has major contributions that indicate the possibility and feasibility of using edge-based intrusion detection on networks where conventional solutions are ineffective due to high complexities affiliated with network activities.

2 Literature Review

The current rapid integration of Internet of Things (IoT) technology paradigms has significantly impacted the modern Networked Habitat environment by ensuring the provision of necessary sensing, automated, and remote control capabilities on a large scale for residential, industrial,

and business environments. Despite the benefits brought about by IoT technology, it poses the most challenges in the secured environment, considering the diversity, low processing, and long installation periods for IoT devices. This is mainly because most of the IoT devices are linked to the internet without incorporating the essential functionality for security and, once installed, they remain unupgraded.

Intrusion Detection Systems (IDS) has been an area of active research for many years as it is considered an imperative part of network security designs. A broad classification of intrusion detection systems has been proposed by Scarfone and Mell that varies from Host-based and Network-based techniques, as well as Signature-based and Anomaly-based techniques for intrusion detection [1]. Host-based intrusion detection systems focus on system-level events, whereas network-based intrusion detection systems focus on network events occurring at a network level. However, host-based intrusion detection is not preferred for IoT environments because of processing power, lack of adequate memory, and many commercial IoT platforms remaining closed-source. Therefore, network-based intrusion detection has been identified as a promising technique.

Detection methods used in intrusion detection systems can be broadly classified as signature or anomaly-based. The signature-based systems use a set of pre-defined rules specifying known attack signatures, which may be in the form of defined packets, protocol, or scanning activity. The methods are popular because they are easy to understand, provide low false positives, and are suitable for real-time monitoring. The major drawback of these methods is their lack of ability to recognize new or unfamiliar threats. The other category, namely the anomaly-based systems, will attempt to recognize anomalies by creating a baseline model using statistical or machine-learning concepts. Though the proposed methods have the potential to recognize unfamiliar threats, large amounts of training are necessary, and large numbers of false positives may be experienced [2]. These factors become more problematic within IoT networks with dynamic and unpredictable traffic.

Some research papers bring out the point that in a constrained resource situation, the simplicity of operation of signature-based methods makes them an attractive and viable alternative. According to research carried out by Sommer and Paxson, the lack of simplicity in operation and interpretation in the case of machine learning methods makes them less attractive to adopt in the real world [2]. This reiterates the point that signature methods are to be preferred in the case of scanning and protocol probing activities.

Edge computing has attracted much attention as a solution for addressing scalability, latency, and privacy issues associated with IoT networks. Edge computing architectures can process data closer to its sources, thereby reducing their dependence on cloud computing. Atzori et al. pointed out that architectural simplicity is a huge issue when it comes to IoT networks. At the same time, Atzori et al. considered distributed processing crucial to making these architectures scalable and responsive [3]. From a security viewpoint, edge computing-based intrusion detection can enhance network surveillance closer to the IoT network.

The study of the security and privacy concerns of the IoT in a distributed environment conducted by Roman et al. is ongoing, and the performance of the centralized security mechanism does not address the special character of the IoT environment [4]. The solution proposed by

the authors is the combination of the use of localized security mechanisms, independent and adaptable to the environment. This explains the focus of attention on the exploration of the application of Lightweight Intrusion Detection Systems operating at the edge, independent of central and powerful processing units.

The advent of cost-effective single board computing solutions like Raspberry Pi has given rise to a novel and promising edge computing solution for security monitoring. The power efficiency, miniature size, and sufficient processing capabilities of the system encourage its use for uninterrupted, around-the-clock operation, even in a small environment. The effectiveness of the Raspberry Pi system for network monitoring, data acquisition, and simple analytic tasks has already been proven in the past works [5]. Therefore, a practical solution for inexpensive security services, beyond the office environment, utilizing this combined with open-source solutions, will be forthcoming.

The role of open-source intrusion detection systems should not be underestimated for the mentioned purposes. The project Suricata [6] is the most popular open-source project for the intrusion detection and prevention system. The system possesses the feature of multi-threading for packet processing, possesses the comprehensive analysis of packets, and retains the structured logging mechanism. The factor that allows applying the system to its full extent, along with analysts from the community, makes the product very attractive for the application either for the purpose of the research or for other uses. The presence of the standardized output formats, like the generation of the result in the form of the JSON file, enables the simplicity of the alert processing.

The testing process of the Intrusion Detection System might also require the utilization of the controlled attack tools that will allow the assessment of the accuracy levels of the detection process. The study carried out by Lyon on Nmap scan techniques provides a rich explanation of scan techniques that have been employed by the attackers during the reconnaissance scan process [7]. The techniques employed by Nmap scan have been widely utilized for research purposes since they can be modified to develop an attack scenario.

Though enormous advances have been made in research regarding the security concerns associated with the IoT, there also exist some open research issues, which are yet to be resolved. With the increasing trend observed in the encrypted traffic, it is becoming more and more difficult to solely depend on the payload-based detection method, such that a new detection method is required to be used, which considers the flow or metadata intelligence. There also exists a need to maintain a trade-off between the accuracy and the decrease in the number of false positives, especially while handling more complex scenarios, including specific traffic patterns.

To finish, it could be concluded that there already is existing literature that is strong enough to provide a good foundation on the application of a network-based NID system in the context of the IoT environment, mainly through the deployment on the edge through low-cost systems and open-source software. Firstly, it could be presumed that there already is existing interdependency on the detection functionality, complexity, and system resources. Mainly through the application of the existing background, it seems that this paper provides a good application through the deployment on the edge and signature detection.

3 Scientific Background and Up-to-Date Knowledge

The massive adoption of IoT devices across the residential, industrial, and business settings has significantly changed the way current networks are managed, as they now enable a huge amount of data to be collected[3]. Although there are benefits associated with the adoption of IoT devices, there are major challenges associated with IoT systems, which act as a major security risk because they are less secured and are less frequently updated after deployment[4]. Additionally, the adoption of IoT devices presents a major challenge to business entities as they reduce the visibility of the business environment.

Intrusion Detection Systems(IDS) solutions generically monitor either network or host activity for the purpose of finding malicious behavior or policy violations. IDS solutions can be broadly categorized into two classes: host-based intrusion detection systems, monitoring activity on individual devices, and network-based intrusion detection systems, analyzing traffic flowing across the network[1]. Host-based approaches are often unrealistic for IoT environments due to limited processing power and memory and the proprietary nature of many devices. Network-based intrusion detection provides a valid alternative that allows central monitoring without the need for any changes to individual IoT devices, thus being better suited for heterogeneous IoT deployments[3].

In the past, traditional intrusion detection systems have been used in business networks through the deployment of specialized hardware and powerful servers. Such systems are not ideally suited for small-scale applications involving IoT because of their associated cost and manageability[5]. However, in recent years, there has been more interest and focus on light-weight intrusion detection mechanisms that are more suited to operation at the edge of networks and near the points of IoT devices. Edge security solutions are designed to enhance scalability and reduce latency[3].

Open-source intrusion detection systems, like Suricata, are being widely adopted owing to their flexibility, extensibility, and community-driven model[6]. Suricata features multi-threaded packet handling, handling of new-generation protocols, and logging facilities, making it amenable to being deployed even on resource-constrained systems, depending upon the settings chosen. When combined with the fact that low-cost SBCs, such as the Raspberry Pi, are available, open-source IDS systems hold a great deal of promise for IoT-based security monitoring[5].

Recently, studies have emerged about the benefits of using edge devices to increase the level of security visibility when enterprise IDSs cannot be implemented[4]. Nevertheless, there are still limitations with regard to achieving a good balance between detection accuracy, CPU consumption, and false alarms[2]. Thus, there is an ongoing need to investigate edge-based IDSs that could definitely offer IoT network security benefits when connected to a wireless networkciteatzori2010iot.

4 Selected Open Topic

The identified open issue addressed in this research work is more particularly concerned with the issue of implementing efficient intrusion detection systems in WiFi-based IoT networks in

domestic, SME, and edge computing setups. Even though in bigger setups highly complex security monitoring infrastructures are commonly used, in smaller IoT networks, no intrusion detection system is implemented in setups where either financial issues or the unavailability of security hardware are of relevance. This has resulted in an important security vulnerability since majority attacking approaches targeting IoT networks invariably involve simple scanning processes that go completely unnoticed.

WiFi networks are an extremely vulnerable yet commonly overlooked attack venue in the context of the IoT. This is because most forms of IoT communication make use of wireless networking, meaning that it is possible to intercept and monitor this communication passively, or to execute active scans against said communication over the wireless network range of the attacker or hacker. In most cases, the security level of such networks is restricted to firewalls or router security, which do not offer a network-level view of activity occurring on the network and thus could be compromised before larger issues arise.

In research, some aspects that might arise when incorporating an intrusion detection system into an IoT environment could be attributed to restrictions on resources within edge devices that might hinder intensive computation processes associated with detection. Moreover, when an alarm level becomes too high, it could pose a challenge for an admin to reap a practical advantage when incorporating an intrusion detection system into a non-enterprise setting.

In this research paper, the solution shall aim to solve the selected open issue by examining a lightweight edge-based approach to an intrusion detection mechanism with the employment of open source tools as well as cost-effective technology. With the emphasis placed on passive monitoring of WiFi transmissions, designing relevant detection rules, and the application of the level of alarm severity, this approach shall endeavor to enable greater awareness with regard to increased security without promoting great overhead cost.

5 Methods and Results

5.1 System Architecture

The design for the intrusion detection system is lightweight, edge-based, and serves the primary function of ensuring WiFi-connected IoT networks are secured. It is implemented using the Raspberry Pi and functions in a passive mode, in which it does not interrupt any functionality on the sensor networks. It also analyzes all traffic on the networks locally without using the cloud.

The architecture is comprised of four major components, which are traffic capture, intrusion detection, alert logging, and alert notification. The capture of WiFi traffic is conducted on the WiFi interface of the Raspberry Pi and channeled to the intrusion detection component. This is where the intrusion detection engine, Suricata, analyzes the packets of traffic that are produced on a real-time basis using pre-defined signature-based detection rules. These rules are basically associated with reconnaissance attacks such as ICMP scans, TCP syn scans, and UDP scans.

Detected events are logged using Suricata's native logging mechanisms. Two different logging formats are in use: first, a human-readable log file that is easy to inspect for a quick view and, second, a structured JSON log file that can be used for further automated post-processing.

The structured logging easily allows for post-processing and alert correlation without having to interface directly with the intrusion detection engine.

Concurrently, a Python-based alerting service works in tandem with Suricata in order to handle newly generated events of alerts. The service passively monitors the structured log file continuously to correlate the alerts by their source and type, applying severity filtering to suppress unnecessary notifications. In the case of high-severity events, summarized e-mail alerts that are generated get delivered to the system administrator. This design ensures that only relevant security events trigger notifications to avoid alert fatigue while maintaining visibility into the potential threats.

There is continuous emphasis on the architecture being simple, modular, and practical. Each module is loosely linked, but all are integrated to constitute the pipeline for monitoring, which makes it possible to extend or alter the system without much affect on other modules.

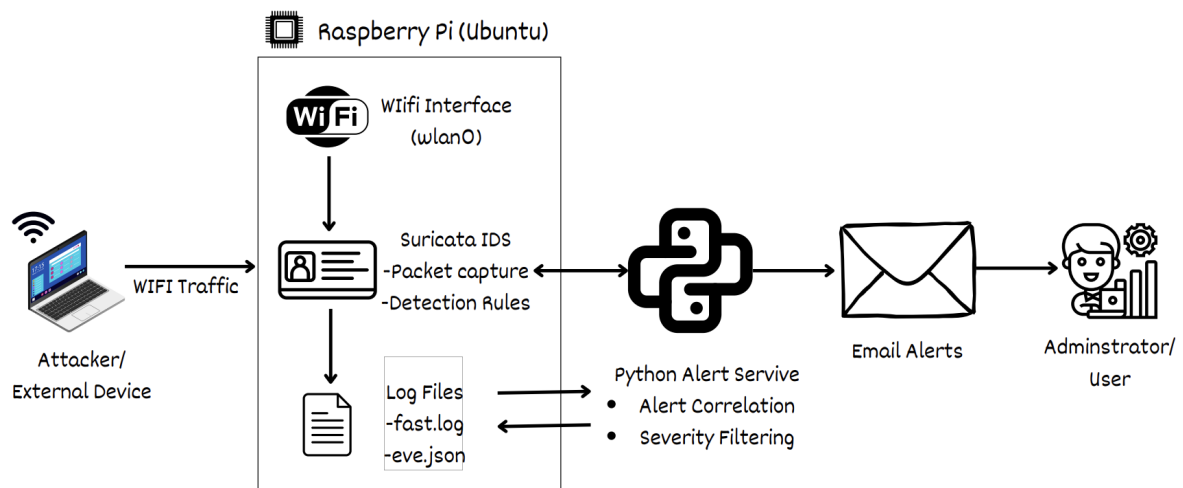


Figure 1: Proposed WiFi-based intrusion detection system architecture.

5.2 Hardware and Software Platform

A Raspberry Pi 5 is used for developing the intrusion detection system, and this is chosen owing to the good compromise between processing speed, power consumption, and associated device costs. The Raspberry Pi will be an appropriate device to make use of in edge-security monitoring, as the device has a processing capacity sufficient enough to handle the traffic present within the network.

It is expected to run on top of Ubuntu 24.04 LTS to ensure that the underlying OS is stable and supported well. The choice of the OS is possible since it supports Suricata, updates are available, and offers a high variety of apps. The wireless interface of the Raspberry Pi is set to sniff the traffic emanating from the local WiFi network.

The intrusion detection system is performed by Suricata version 8. Suricata is an open-source network intrusion detection system that has high performance capability, supports multi-threaded processing of packets, and is compatible with modern network protocols. In terms of packet capture, AF-Packet is utilized. This framework supports interaction with the network interface, which enables an intrusion detection system. This set-up supports real-time traffic

analysis with minimal losses and overhead.

Besides this, a custom Python script is implemented to handle the alerts and notifications. The script prepares the alerts received from the JSON formatted messages of Suricata, creates the new events using the offset technique, and filtered outputs for email notifications. The said service of alert notification is configured as a system service. Due to this, the service automatically starts and runs with no need for human interaction.

This integration of hardware and software components leads to a compact, effective intrusion detection system. The proposed system illustrates that network security monitoring is actually possible by using low-budget hardware components along with open-source solutions, therefore being effective predominantly in a WiFi-based IoT network.

5.3 Traffic Capture and Detection Rules

Network traffic sniffing is done in a passive manner to ensure that the intrusion detection system does not impair the functionality of the network. The Raspberry Pi detects the traffic that flows through the wireless network in a manner that does not involve sending, modifying, or discarding any packets. This becomes very necessary in an IoT network because any lag in the network or any irregularity in the network might affect the IoT devices.

The packets are received in Suricata directly from the wireless network interface using the AF-Packet framework. One of the advantages of the AF-Packet framework is the ability to do the captured packets at a faster rate. This ensures that Suricata is able to use an effective method to handle packets at a faster rate with little or no loss and little processing overhead. This particular method operates at the edge of the network and is able to intercept communication between the IoT device and the computer without changing the configuration setting. Intrusion detection in the proposed system happens through a set of custom-made rules that define network signature-based intrusion attempts. This is often an initial phase for an attempted attack and may be critical for those networks that include IoT components, mainly since attackers use scanning actions in order to identify weak assets before actually attacking. One of the related intrusion attempts that take place through these custom-made rules involves ICMP echo requests. This particular protocol can be regarded as one of the common tools used by attackers in order to see who is present and in what manner network topography is mapped out. Although this protocol by itself is considered an attack attempt, actions can be taken with regard to intrusion attempts through customized rules regarding these protocols.

Second, there are scans related to TCP SYN scanning activity, which is common to attackers looking for available ports and services running on target machines. Detection of this scanning activity is based on the TCP packet with the SYN flag set, together with some threshold-based logic to detect repeated connection attempts from the same address within a certain time frame. This helps avoid false positives coming from actual connection attempts.

The system also tackles UDP scan traffic, which has always presented a challenge in attempting to detect such traffic owing to its nature of being connectionless yet in the presence of some background legitimate traffics such as DNS queries. The system therefore ensures that thresholding is handled in an effective manner to detect UDP scan traffics over a port set owing to such challenges.

The optimization of the rules for detection has to be carefully monitored to achieve the right compromise between the levels of detection and alert. The threshold parameters have been adjusted to make sure that for any isolated activity, including packets, if it does not engage in a scanning session, the alert will never be triggered, and the scanning activity will also not go undetected. The threshold parameters have been adjusted to make sure that the minimum levels of false alarms are achieved. A level of severity will be assigned to each detection rule to indicate the level of risk posed by the scanned activity. TCP and UDP scans are reconnaissance attacks, and the level of risk associated is the highest since the scans are a precursor to other, potentially more serious, attacks or threats. This is denoted by the alert procedure, which is dependent on the definition of the threshold level since above the threshold level, emails are sent to notify the recipient to view the alert threshold level.

5.4 Alert Logging and Notification Mechanism

Each time a detection rule is triggered, Suricata logs the activity using various logging methods. The proposed system makes use of two types of log formats. The first one is the plain text alert log. It supplies a quick and easy-to-read view of events that have been detected. These logs are ideal when it comes to validation of detection mechanisms during the testing phase, apart from being used in the process of removing bugs.

Apart from the text alerts, the other log file Suricata produces is the structured log file in the format of JavaScript Object Notification (JSON). In the structured log file, the metadata about the alerts is well captured, including the source and destination IP addresses, protocol details, rule names, and severity levels. This logged information is of prime importance because it enables automatic processing of the alerts and other analysis tasks to be carried out without the need to directly interact with the intrusion detection system. Structured logging makes the alert pipeline easily extensible to other analysis tasks or incorporating other monitoring services.

To handle the alert notification data in a organized manner, which is generated by the Suricata system, a Python alert notification service is used and implemented within the system. The system is also monitoring the JSON log file constantly for new alert entry additions, which would be used in the creation of alert detail information in the process of sending notifications. By utilizing the entry tracking approach via the offset technique, redundancy in the creation of alert notification messages is eliminated, and standardization is obtained within the alert message notification based on the restart of the system services.

Correlation of alerts can be achieved through the grouping of events based on the source IP address as well as the type of alert. By this means, the problem of redundancy is addressed since the alerts produced by scanning processes will be provided in a summarized manner. Unlike the case where the user is provided with every single packet that triggered the rule, the user will be provided with a summary of the events within a certain time frame, thus making it quite user-friendly since there is a scenario where multiple scanning attempts are performed within a very short time.

In order to further limit the noise level of the alerts, the solution implements a severity level for filtering before the notification process. This warning level is associated with the level of

risk of the discovered activity. Activities initiated by a lower level of warnings could be part of background traffic, while they are recorded for logging purposes only. Activities initiated by a high level of warnings are seen as actions and provide warnings to the administrator.

When there are occurrences of one or more severe alerts, the alerting service sends out a summary notification through an emailed message. The alerted mail contains informative details such as the name of the detected activity, the source IP address, as well as the time span when the activities took place. The choice of using emailed notifications was based on the simplicity of the method as well as its applicability to small environments where the administrators lack access to central security alerting stations. The alert system strikes a balance regarding the visibility and feasibility of the notification.

5.5 Experimental Evaluation

The experiment of the proposed intrusion detection mechanism was performed using simulated attacks in order to imitate real-world interactions on a network. The experiments were intended to evaluate the precision of the detection, watch the notification procedure, and see the feasibility of the mechanism in its actual context.

Attack traffic was performed using Nmap scan tool and came from an external client that was joined to the same Wi-Fi network. Reconnaissance scans used in simulation included three different types of scans, which involved ICMP echo scans to test the availability of target hosts, TCP SYN scans used to identify the ports and corresponding services available on the target host machines, and UDP scans involving scans of multiples of destination ports. In this context, the scans provide a well-balanced attack simulation because the scans are realistic methods of attack used by hackers to carry out the reconnaissance phase of host network attack events.

During the experiments, the Raspberry Pi configured to operate as a passive monitors system only, which meant that the Raspberry Pi had the ability to monitor the network traffic without interrupting or affecting it. Besides the network traffic that was part of the attack scenarios, general network traffic consisting of web browsing and DNS queries was included to allow assessment of system performance under these conditions.

The detection behavior evident during the testing process was able to identify the sample reconnaissance activities efficiently. Echo requests of the ICMP type have been identified as probes, and the activity of SYN scans using TCP and UDP protocols generated alerts once threshold values have been exceeded. This ensured that legitimate or isolated packets did not activate false alerts through the threshold logic incorporated in the detection routine. Alerts produced by Suricata have been processed by the alerting service and recorded successfully.

On the whole, the empirical assessment has successfully verified that the developed system can efficiently identify the typical network reconnaissance attack behavior and keep the system running stably without increasing the false alert notifications.

```
pkz@pkz:~$ sudo tail -10 /var/log/suricata/fast.log
[sudo] password for pkz:
12/31/2025-14:48:57.431951  [**] [1:1000003:3] UDP Scan Detected [**] [Classification: Detection of
a Network Scan] [Priority: 1] {UDP} 192.168.0.193:48354 -> 192.168.0.137:445
12/31/2025-14:49:06.033658  [**] [1:1000003:3] UDP Scan Detected [**] [Classification: Detection of
a Network Scan] [Priority: 1] {UDP} 192.168.0.193:48356 -> 192.168.0.137:59846
12/31/2025-14:49:11.329105  [**] [1:1000003:3] UDP Scan Detected [**] [Classification: Detection of
a Network Scan] [Priority: 1] {UDP} 192.168.0.193:48348 -> 192.168.0.137:49186
12/31/2025-14:51:06.214282  [**] [1:1000001:2] ICMP Echo Request Detected [**] [Classification: Gene
ric ICMP event] [Priority: 2] {ICMP} 192.168.0.193:8 -> 192.168.0.137:0
12/31/2025-14:51:07.218997  [**] [1:1000001:2] ICMP Echo Request Detected [**] [Classification: Gene
ric ICMP event] [Priority: 2] {ICMP} 192.168.0.193:8 -> 192.168.0.137:0
12/31/2025-14:51:08.229947  [**] [1:1000001:2] ICMP Echo Request Detected [**] [Classification: Gene
ric ICMP event] [Priority: 2] {ICMP} 192.168.0.193:8 -> 192.168.0.137:0
12/31/2025-14:51:09.223273  [**] [1:1000001:2] ICMP Echo Request Detected [**] [Classification: Gene
ric ICMP event] [Priority: 2] {ICMP} 192.168.0.193:8 -> 192.168.0.137:0
12/31/2025-14:51:10.228818  [**] [1:1000001:2] ICMP Echo Request Detected [**] [Classification: Gene
ric ICMP event] [Priority: 2] {ICMP} 192.168.0.193:8 -> 192.168.0.137:0
12/31/2025-14:51:21.152497  [**] [1:1000002:3] TCP SYN Scan Detected [**] [Classification: Detection
of a Network Scan] [Priority: 1] {TCP} 192.168.0.193:33258 -> 192.168.0.137:445
12/31/2025-14:51:22.206079  [**] [1:1000003:3] UDP Scan Detected [**] [Classification: Detection of
a Network Scan] [Priority: 1] {UDP} 192.168.0.193:44237 -> 192.168.0.137:9001
```

Figure 2: Example intrusion detection alerts recorded by Suricata in fast.log during TCP and UDP scan activity.

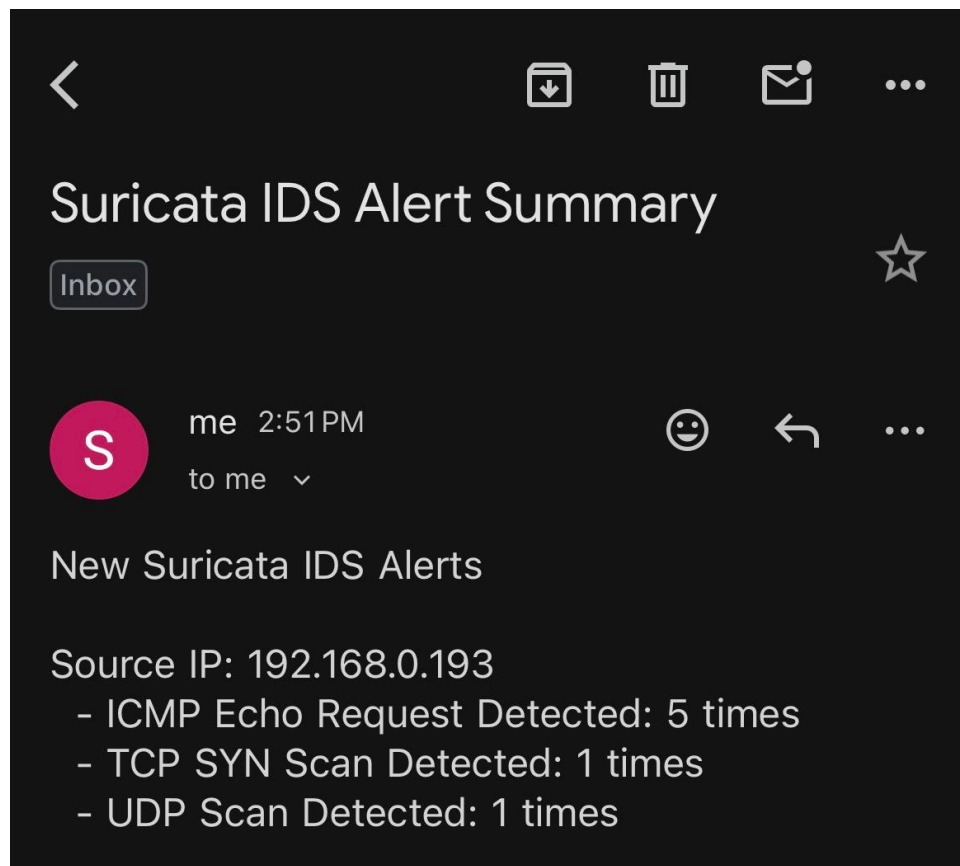


Figure 3: Email notification generated by the IDS alerting service.

6 Open Questions and Future Work

1.How does encrypted network traffic impact intrusion detection in IoT environments? The other challenge in network-based NIDS systems will be the increasing use of encryption in network communication. With the increasing use of encryption in network proto-

cols, the possibility of observing the payload of the packets will be restricted. Therefore, the use of the payload observation method in detecting intrusion will be restricted in the future. Exploring other methods such as the use of traffic metadata, traffic patterns, or statistical patterns will be a field of research in the future.

2.How can the risk of false positives be reduced in long-term deployments?

Although the proposed system integrates a thresholding technique for the reduction of alert noise and filtering based on severity, false alert occurrences can happen under some network conditions. Long-term real-world exposure may reveal other benign network patterns as well that currently cause alerts to go off. Some possible future integrations can include an adaptive thresholding technique.

3.Is there an opportunity to apply the anomaly-based detection within the system? The current system relies on the signature recognition method, which proves to be fairly efficient in the aspect of recognizing an attack through the use of recognized attack signatures, but this proves to be inefficient in the aspect of recognizing an unknown threat. The use of methods that provide anomaly identification techniques, such as statistical methods and machine learning techniques, would prove to be an addition to the aspect of recognizing an unknown threat. Again, challenges emerge in the aspect of model training.

4.Can the existing system be scaled up for use within the larger or mesh IoT environment? Currently, it has been employed as a single point for monitoring intrusions within the local Wi-Fi network. The extension of the system to integrate multiple sensors is an area of great relevance when it comes to the future. The system will be responsible for aggregating the alert messages to form an overall perspective of the events taking place across various parts of the networks.

5.What might the role of SIEM integration be in future developments of the system? Integration with Security Information and Event Management systems provides an opportunity for the analysis of warnings and monitoring. The export of alert data to a Security Information and Event Management system will provide the possibility for the analysis of historical data. The integration will expand the range of using the system at a semi-professional and professional level.

7 Conclusion

This paper discussed the design, testing, and evaluation of an efficient WiFi-based passive IoT intrusion detection system through a Raspberry Pi platform based on a Suricata engine. The development was based on an edge technology approach, making it an effectual passive solution requiring no changes within the WLAN devices used by different IoT applications.

This proposed approach is able to successfully identify general scans of the network, such as ICMP ping scans, SYN scans, and UDP scans. By using adjustable detection rules and alerting priorities, general scans of the network have been successfully identified while avoiding unnecessary alerts and the consequent consumption of network and processing resources by the Raspberry Pi platform.

In terms of its applications and implications, this work clearly indicates that it is entirely possible to monitor the security of computer networks by using relatively inexpensive hardware and free operating code. Of course, this is particularly applicable especially in homes and other smaller organizations and institutions where other more sophisticated systems and methods would not make too much financial and technical sense.

The areas where this research could be extended include the incorporation of anomaly-driven detection mechanisms to enable better detection rates for unfamiliar threats, improved analysis of encrypted traffic using flow-level analysis, and implementing various distributed sensors and centralized alert analysis. These areas can serve as promising extensions to pursue and develop upon the practical applications demonstrated by this proposed intrusion detection solution.

References

- [1] K. Scarfone and P. Mell, "Guide to intrusion detection and prevention systems," NIST, Tech. Rep., 2007. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-94.pdf>.
- [2] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," *IEEE Symposium on Security and Privacy*, 2010. DOI: [10.1109/SP.2010.25](https://doi.org/10.1109/SP.2010.25).
- [3] L. Atzori, A. Iera, and G. Morabito, "The internet of things: A survey," *Computer Networks*, 2010. DOI: [10.1016/j.comnet.2010.05.010](https://doi.org/10.1016/j.comnet.2010.05.010).
- [4] R. Roman, J. Zhou, and J. Lopez, "On the features and challenges of security and privacy in distributed iot," *Computer Networks*, 2013. DOI: [10.1016/j.comnet.2012.12.018](https://doi.org/10.1016/j.comnet.2012.12.018).
- [5] A. Behl, *Cybersecurity and Cyberwar*. Oxford University Press, 2017.
- [6] *Suricata user guide*, Open Information Security Foundation, 2024. [Online]. Available: <https://suricata.io/documentation/>.
- [7] G. Lyon, *Nmap Network Scanning*. Insecure.org, 2009. [Online]. Available: <https://nmap.org/book/>.